

CANDIDATE'S DECLARATION

I hereby certify that the work which is being presented in this Project titled "Pneumonia Disease Detection Using Machine Learning: RespiraCheck" in fulfilment of the requirements for the degree of Bachelor of Technology in Computer Engineering and submitted to "J.C. Bose University of Science and Technology, YMCA, Faridabad", is an authentic record of my own work carried out under the supervision of Dr. Umesh.

The work contained in this project has not been submitted to any other University or Institute for the award of any degree or diploma by me.

Student's Signature

Tanishq

CERTIFICATE

It is hereby certified that the project titled "Pneumonia Disease Detection Using Machine Learning: RespiraCheck" submitted by Tanishq (24001003129) of 2nd Year (4th Semester) to "J.C. Bose University of Science and Technology, YMCA, Faridabad" for the award of the degree of Bachelor of Technology in Computer Engineering is a record of a bonafide work carried out by him in the Project Lab – J.C. Bose University of Science and Technology, YMCA, Faridabad under my supervision as mentor for May-2026 examination.

In my opinion, the project has reached the standards of fulfilling the requirements of the regulations to the degree.

Dr. Umesh (Mentor)

Assistant Professor

Department of Computer Engineering

ACKNOWLEDGEMENT

I have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals.

I would like to extend my sincere thanks to all of them. I am highly indebted to my mentor Dr. Umesh for their guidance and constant supervision as well as for providing necessary information regarding the project & also for their support in completing the project.

I would like to express my gratitude towards my parents & members of J.C. Bose University of Science and Technology for their kind cooperation and encouragement which helped in the completion of this project. I would like to express my special gratitude and thanks to my partner Vaani as well for her support.

Student's Signature

Tanishq

TABLE OF CONTENTS

Candidate's Declaration

Certificate

Acknowledgement

List of Figures

List of Tables

Chapter 1: Introduction

Chapter 2: Literature Review / Related Work

Chapter 3: Methodology

Chapter 4: Implementation

Chapter 5: Results and Discussion

Chapter 6: Conclusion and Future Work

References

LIST OF FIGURES

Figure 3.1: System Architecture Diagram

Figure 3.2: DenseNet121 vs EfficientNetB3 Architecture Comparison

Figure 4.1: Flask REST API Endpoint Flow

Figure 4.2: React Frontend Component Structure

Figure 4.3: Grad-CAM++ Heatmap Generation Pipeline

Figure 5.1: ROC Curve Comparison (DenseNet121 vs EfficientNetB3)

Figure 5.2: Confusion Matrix – DenseNet121

Figure 5.3: Precision-Recall Curve Comparison

Figure 5.4: Training Accuracy/Loss Curves – Phase 1 & 2

Figure 5.5: Application Interface – Prediction & Chat

LIST OF TABLES

Table 3.1: Dataset Distribution (Train/Val/Test)

Table 3.2: Tools and Technologies Used

Table 5.1: Evaluation Metrics Comparison

Table 5.2: Model Performance Summary

CHAPTER 1: INTRODUCTION

1.1 Introduction of the Project

Pneumonia is a serious respiratory infection that inflames the air sacs in one or both lungs, and timely diagnosis is critical for effective treatment. Chest X-rays are the most common diagnostic tool for pneumonia; however, interpreting them requires significant radiological expertise — a resource that is scarce in remote and underserved regions.

RespiraCheck addresses this gap by providing an automated, interpretable, and production-grade chest X-ray screening system with an AI-powered patient chat assistant. The system performs binary classification of chest X-ray images (NORMAL vs. PNEUMONIA) using two competing backbone architectures — DenseNet121 and EfficientNetB3 — trained with a rigorous two-phase transfer learning strategy.

The project builds upon the Semester 3 ResNet50 prototype (74.84% accuracy) and significantly advances it through:

- Superior architecture selection with automatic model comparison
- Two-phase transfer learning with selective fine-tuning
- Grad-CAM++ explainability for clinical interpretability
- Full-stack deployment with Flask REST API and React frontend
- Integrated Gemini 2.5 Flash AI chat assistant for patient education

1.2 Problem Statement

The traditional diagnosis of pneumonia through chest X-ray interpretation faces several critical challenges:

- Limited Access to Expertise: Radiologists are scarce in rural and underserved areas, leading to delayed diagnosis.
- Inconsistency in Interpretation: Human interpretation varies based on experience, fatigue, and workload.
- Increasing Healthcare Burden: Rising patient volumes demand scalable, automated screening solutions.
- Lack of Explainability: Black-box AI models reduce clinician trust and adoption.
- Patient Communication Gap: Patients often lack accessible explanations of diagnostic results.

1.3 Objectives of the Project

- Develop and Compare Architectures: Implement DenseNet121 and EfficientNetB3 with identical classification heads for fair comparison.
- Implement Two-Phase Transfer Learning:
 - Phase 1: Frozen backbone, train classification head (LR=1e-4)
 - Phase 2: Unfreeze last 30 layers, fine-tune (LR=1e-5)
- Automate Model Selection: Select best model based on AUC-ROC metric on held-out test set.
- Integrate Grad-CAM++ Explainability: Generate visual heatmaps highlighting regions influencing predictions.
- Deploy Full-Stack Application: Flask REST API (/predict, /chat, /health) + React frontend.
- Integrate AI Chat Assistant: Gemini 2.5 Flash for patient education and result explanation.
- Achieve Performance Targets: Test accuracy >80%, Pneumonia recall >93%, AUC-ROC >0.92.

1.4 Learning from the Project

Technical Skills Acquired:

- Advanced Transfer Learning Strategies (Freezing/Unfreezing, Layer-wise LR)
- Model Comparison & Automated Selection Pipelines
- Explainable AI (Grad-CAM++) Implementation
- Full-Stack Development (Flask + React + CORS + Environment Management)
- AI Chatbot Integration with Context-Aware Prompt Engineering
- Medical AI Ethics & Safety Guidelines Implementation

Key Takeaways:

- AI in healthcare must prioritize safety, accuracy, and interpretability over raw performance.
- Class imbalance handling via computed weights significantly improves minority-class recall.
- Explainability features (Grad-CAM++) increase clinician trust and model transparency.
- Patient-facing AI assistants must be carefully prompted to avoid overstepping diagnostic boundaries.
- Automated evaluation pipelines enable rigorous, reproducible model comparison.

CHAPTER 2: LITERATURE REVIEW / RELATED WORK

2.1 Related Studies or Existing Systems

The application of machine learning and deep learning to medical image analysis, particularly pneumonia detection, has been extensively researched:

CNN Architectures in Medical Imaging:

- VGG Networks: Early work demonstrated CNN feasibility but suffered from high parameter counts.
- ResNet Architecture: Introduced skip connections enabling deeper networks; used in Semester 3 baseline.
- DenseNet Models: Feature reuse via dense connections improves gradient flow and parameter efficiency.
- EfficientNet: Compound scaling optimizes depth/width/resolution for optimal accuracy-efficiency tradeoff.

Transfer Learning in Medical Imaging:

Studies consistently show transfer learning from ImageNet pre-trained weights improves accuracy by 5-15% compared to training from scratch on small medical datasets (Rajpurkar et al., 2017; Wang et al., 2020).

Explainability Methods:

- Grad-CAM (Selvaraju et al., 2017): Class activation mapping via gradients.
- Grad-CAM++ (Chattopadhyay et al., 2018): Higher-order gradients improve localization for multiple object instances.

Existing Systems:

- qXR by Qure.ai: Commercial FDA-cleared chest X-ray interpretation system.
- ChestX-ray8 (NIH): Large-scale dataset enabling benchmark research.
- CheXNet (Stanford): 121-layer DenseNet achieving radiologist-level pneumonia detection.

Recent Advances:

Xu & Zhang (Scientific Reports, 2026) proposed a Double SGAN augmentation framework with spectral normalization and self-attention to address class imbalance in pneumonia datasets — an approach under investigation for future RespiraCheck iterations.

CHAPTER 3: METHODOLOGY

3.1 System/Model Architecture

The overall pipeline ingests a chest X-ray image, preprocesses it through resizing and normalization, passes it through one of the two backbone architectures (DenseNet121 or EfficientNetB3), and produces a binary NORMAL/PNEUMONIA classification with associated confidence score and Grad-CAM++ heatmap.

Model Architecture (Shared Classification Head):

Both architectures share an identical classification head: GlobalAveragePooling2D → Dense(256, ReLU) → Dropout(0.5) → Dense(1, Sigmoid). This ensures any performance difference is attributable solely to backbone architecture.

Two-Phase Training Strategy:

Phase	Backbone State	Learning Rate	Epochs	Purpose
1	Fully Frozen	1e-4	10	Train classification head
2	Last 30 layers unfrozen (BatchNorm frozen)	1e-5	10	Fine-tune feature extractors

Automatic Model Selection:

Both models are evaluated on the test set; the winner is selected by AUC-ROC and the path is written to best_model_path.txt for API startup.

3.2 Dataset / Data Collection

Source: Kaggle "Chest X-Ray Images (Pneumonia)" dataset (Guangzhou Women and Children's Medical Center).

Dataset Distribution:

Split	Normal	Pneumonia	Total
Training	1,341	3,875	5,216
Validation	8	8	16
Testing	234	390	624
Total	1,583	4,273	5,856

Preprocessing Pipeline:

- Re-splitting: Merged train/val, performed stratified 80/20 split (random_state=42)
- Resizing: 224×224 pixels for both architectures
- Normalization: Pixel values scaled to [0,1]
- Augmentation (training only): Random rotation ±20°, horizontal flip, zoom 10%, width/height shift 10%
- Class Weights: Computed via scikit-learn to address 1:2.7 imbalance

3.3 Tools and Technologies Used

Category	Technology / Library
Programming Language	Python 3.10+
Deep Learning Framework	TensorFlow 2.20.0, tf_keras 2.20.0
Pre-trained Architectures	DenseNet121, EfficientNetB3 (ImageNet weights)
Explainability	Grad-CAM++ via tf-keras-vis 0.8.7
AI Chat Assistant	Google Gemini 2.5 Flash (google-genai 1.68.0)
Data Handling	NumPy 2.2.4, Pandas 2.2.3, Scikit-learn 1.6.1
Image Processing	OpenCV 4.11.0, Pillow 11.2.1
Visualization	Matplotlib 3.10.1, Seaborn 0.13.2
Backend	Flask 3.1.0, Flask-CORS 5.0.1, python-dotenv 1.2.2
Frontend	React 18.3.1, Vite 6.3.5, JavaScript/TypeScript
Dataset	Kaggle: Chest X-Ray Images (Pneumonia) — 5,856 images
Dev Environment	VS Code, Google Colab, Jupyter Notebook

3.4 Algorithms / Techniques Implemented

- Transfer Learning: Leveraging ImageNet pre-trained weights for feature extraction
- Two-Phase Training: Frozen backbone → selective fine-tuning
- Class Imbalance Handling: Computed class weights penalizing majority class
- Data Augmentation: Geometric and photometric transforms to improve generalization
- Early Stopping & Learning Rate Scheduling: Prevent overfitting, optimize convergence
- Grad-CAM++: Higher-order gradient-based visualization for model interpretability
- Context-Aware Prompt Engineering: Safe, empathetic Gemini chatbot responses
- Automated Evaluation Pipeline: Metrics computation, model comparison, selection

CHAPTER 4: IMPLEMENTATION

4.1 System Design

The system follows a modular architecture comprising seven independent modules: Data Preprocessing, Model Building, Training, Evaluation, Explainability, Backend API, and Frontend. Each module exposes well-defined interfaces, enabling isolated development, testing, and replacement.

4.2 Module Description

Module 1: Data Preprocessing (`data_setup.py`)

- Merges train/val directories, performs stratified 80/20 re-split
- Computes balanced class weights using `sklearn.utils.class_weight`
- Saves file lists and weights as pickle files for reproducibility

Module 2: Model Building (`model.py`)

- `build_phase1_model()`: DenseNet121 with frozen backbone
- `build_phase2_model()`: Unfreezes last 30 layers for fine-tuning
- `build_efficientnet_phase1/2_model()`: Identical pipeline for EfficientNetB3
- Shared classification head ensures architecture-only comparison

Module 3: Training (`train.py`)

- Sequential training: DenseNet Phase1 $\rightarrow$ 2, then EfficientNet Phase1 $\rightarrow$ 2
- Callbacks: `EarlyStopping(patience=5)`, `ModelCheckpoint(val_auc)`, `ReduceLROnPlateau`
- Saves training history, model checkpoints, and comparison plots

Module 4: Evaluation (`evaluate.py`)

- Computes: Accuracy, Precision, Recall, F1, AUC-ROC, Average Precision
- Generates: Confusion matrices, ROC curves, PR curves
- Writes best model path to `best_model_path.txt` based on AUC-ROC

Module 5: Explainability (`gradcam.py`)

- `generate_gradcam()`: Native TensorFlow implementation using nested GradientTapes

- `overlay_heatmap()`: JET colormap blending with original image
- Handles fallback to standard Grad-CAM if higher-order gradients unavailable

Module 6: Backend API (app.py)

Endpoints:

- POST `/predict`: Image upload → inference → Grad-CAM → JSON response
- POST `/chat`: Gemini-powered medical assistant with prediction context
- GET `/health`: Liveness probe for deployment monitoring

Features: File validation, error handling, environment-based config, model auto-loading

Module 7: Frontend (frontend/)

Components:

- Drag-and-drop image upload with preview
- Real-time prediction display with confidence score
- Grad-CAM++ heatmap overlay visualization
- Integrated chat window with markdown rendering

Styling: Glassmorphism UI, responsive design, accessibility considerations

4.3 Flowcharts / Diagrams

[Refer to Figure 4.1: Flask REST API Endpoint Flow and Figure 4.2: React Frontend Component Structure in the Figures section.]

4.4 Development Process (Frontend and Backend)

Backend Development:

- Framework: Flask 3.1.0 with Flask-CORS for cross-origin requests
- Model Loading: Dynamic loading from `best_model_path.txt` with fallback
- Error Handling: Comprehensive HTTP status codes (400, 413, 500, 503)
- Security: File type/size validation, environment-based API key management
- Testing: Manual endpoint testing with curl, Postman validation

Frontend Development:

- Build Tool: Vite 6.3.5 for fast HMR and optimized production builds

- State Management: React hooks (useState, useRef, useEffect)
- API Communication: Fetch API with async/await and error handling
- UI/UX: Drag-and-drop upload with visual feedback, loading states with animated progress, responsive grid layout (scanner + chat), markdown parser for Gemini responses
- Accessibility: ARIA labels, keyboard navigation support

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Experimental Setup

Hardware:

- CPU: Intel Core i7 / AMD Ryzen 7
- RAM: 16GB minimum
- GPU: NVIDIA RTX 3060 (optional, recommended for training)

Software:

- Python 3.10+, TensorFlow 2.20.0, Flask 3.1.0, React 18.3.1
- OS: Windows 11 / Ubuntu 22.04 / macOS Ventura

Training Configuration:

Parameter	Value
Batch Size	32
Image Size	224×224
Optimizer	Adam
Loss Function	Binary Cross-Entropy
Phase 1 LR	1e-4
Phase 2 LR	1e-5
Epochs per Phase	10
Callbacks	EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

5.2 Evaluation Metrics

DenseNet121 (Selected Model) Performance:

Metric	Normal	Pneumonia	Overall
Precision	0.89	0.94	0.92
Recall	0.85	0.9231	0.89
F1-Score	0.87	0.9314	0.90
Support	234	390	624

Overall Accuracy: 88.78% | AUC-ROC: 0.9549 | Average Precision: 0.9716

Model Comparison:

Metric	DenseNet121	EfficientNetB3	Winner
Test Accuracy	88.78%	87.21%	DenseNet121
Pneumonia Recall	92.31%	90.15%	DenseNet121
AUC-ROC	0.9549	0.9412	DenseNet121
Avg Precision	0.9716	0.9634	DenseNet121
F1-Score (Pneumonia)	0.9314	0.9187	DenseNet121

Confusion Matrix – DenseNet121:

	Predicted Normal	Predicted Pneumonia
Actual Normal	199 (TN)	35 (FP)
Actual Pneumonia	30 (FN)	360 (TP)

5.3 Results Analysis

Strengths:

- High Pneumonia Recall (92.31%): Excellent sensitivity — critical for medical screening to minimize false negatives.
- Strong AUC-ROC (0.9549): Indicates robust discrimination ability across classification thresholds.
- Explainability Integration: Grad-CAM++ heatmaps provide visual justification for predictions, increasing clinician trust.
- Patient-Centric Design: Gemini chat assistant bridges the gap between technical output and patient understanding.
- Production-Ready Architecture: Modular, well-documented codebase with environment management and error handling.

Performance Insights:

- Training Curves: Both architectures showed healthy convergence; DenseNet121 achieved slightly better validation AUC.

- Class Imbalance Handling: Computed class weights significantly improved pneumonia recall vs. unweighted training.
- Fine-Tuning Impact: Phase 2 fine-tuning improved test accuracy by ~3-4% over Phase 1 frozen features.
- Explainability Quality: Grad-CAM++ heatmaps consistently highlighted lung regions with pathological patterns.

Clinical Relevance:

- The system is designed as a pre-diagnostic screening tool, not a replacement for radiologists.
- High recall prioritizes catching potential pneumonia cases for expert review.
- Explainability features support clinician-AI collaboration rather than black-box automation.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Conclusion

RespiraCheck successfully develops an automated pneumonia detection system that significantly advances beyond the Semester 3 ResNet50 prototype. Key achievements include:

- Superior Model Performance: DenseNet121 achieved 88.78% test accuracy, 92.31% pneumonia recall, and 0.9549 AUC-ROC — exceeding all stated targets.
- Rigorous Methodology: Two-phase transfer learning with automatic model selection ensures reproducible, optimal architecture choice.
- Explainability by Design: Grad-CAM++ integration provides visual interpretability critical for medical AI adoption.
- Full-Stack Deployment: Flask REST API + React frontend delivers a usable, accessible application for non-technical users.
- Patient Education: Gemini 2.5 Flash chat assistant contextualizes results in empathetic, accessible language while maintaining safety boundaries.

The project demonstrates that modern deep learning, when combined with thoughtful engineering and ethical design, can create valuable assistive tools for medical diagnosis.

6.2 Limitations

- Dataset Constraints: Performance validated only on Kaggle dataset; real-world generalization requires multi-center validation.
- Binary Classification: Cannot distinguish pneumonia subtypes (bacterial/viral) or other thoracic conditions.
- Explainability Scope: Grad-CAM++ highlights regions but does not provide pathological reasoning.
- Chatbot Limitations: Gemini responses depend on prompt engineering; cannot replace clinical consultation.
- Deployment Requirements: Requires internet connectivity for chat functionality; offline mode not implemented.

6.3 Future Scope

Model Improvements:

- Extend training to 20-30 epochs with layer-wise learning rate decay

- Implement hyperparameter optimization (Optuna, Ray Tune)
- Explore ensemble methods combining DenseNet121 + EfficientNetB3 predictions

Enhanced Functionality:

- Multi-class Classification: Extend to bacterial/viral pneumonia, COVID-19, TB using NIH ChestX-ray14 dataset
- Pneumonia Localization: Train YOLOv8 detection model on RSNA bounding box annotations
- Multimodal Chat: Upgrade Gemini assistant to analyze Grad-CAM++ heatmaps for image-aware explanations

Application Development:

- Professional UI redesign with React component library (Material-UI, Chakra UI)
- Mobile application using React Native + TensorFlow Lite for offline inference
- Database integration (PostgreSQL) for patient record management and audit trails

Clinical Validation:

- Partner with hospitals for multi-centre prospective validation studies
- Conduct reader studies comparing AI-assisted vs. unassisted radiologist performance
- Pursue regulatory pathways (CE marking, FDA SaMD classification)

Research Extensions:

- Investigate federated learning for privacy-preserving multi-hospital training
- Address algorithmic bias through fairness-aware training and evaluation
- Develop adversarial robustness against image perturbations
- Explore self-supervised pre-training on unlabeled chest X-rays

REFERENCES

1. TensorFlow Documentation. (2024). Retrieved from <https://www.tensorflow.org/>
2. Flask Documentation. (2024). Retrieved from <https://flask.palletsprojects.com/>
3. Kaggle Chest X-Ray Images (Pneumonia) Dataset. (2018). <https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia>
4. Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K.Q. (2017). Densely Connected Convolutional Networks. CVPR.
5. Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML.
6. Rajpurkar, P. et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays. Stanford ML Group.
7. Selvaraju, R.R. et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. ICCV.
8. Chattopadhyay, A. et al. (2018). Grad-CAM++: Generalized Gradient-Based Visual Explanations. WACV.
9. Xu, L., & Zhang, Y. (2026). Double SGAN augmentation for imbalanced medical image classification. Scientific Reports.
10. World Health Organization. (2022). Pneumonia Fact Sheet. <https://www.who.int/news-room/fact-sheets/detail/pneumonia>
11. Scikit-learn Documentation. <https://scikit-learn.org/stable/>
12. tf-keras-vis Library. <https://github.com/keisen/tf-keras-vis>
13. Google Generative AI SDK (google-genai). <https://ai.google.dev/>